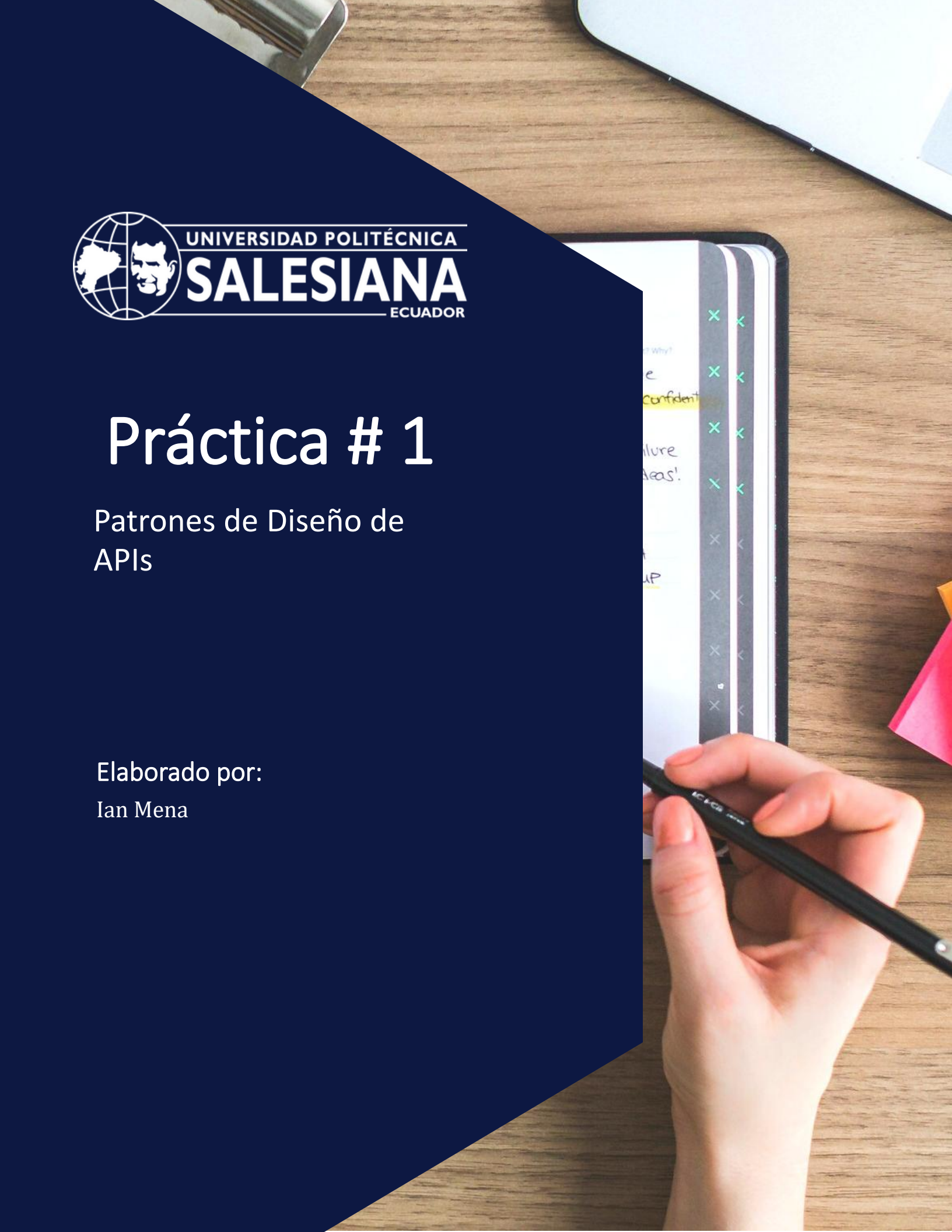


UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

Práctica # 1

Patrones de Diseño de APIs

Elaborado por:

Ian Mena

Actividades

Práctica: **Uso de POSTMAN**

Descripción de la actividad

En la segunda práctica se realizó una exploración de las diferentes APIs que ofrece Google, con el objetivo de comprender cómo se encuentran organizadas, cuáles son sus principales funcionalidades y de qué manera pueden integrarse en aplicaciones externas.

Inicialmente, se revisaron las distintas categorías de APIs disponibles en Google, analizando sus funcionalidades, mecanismos de autenticación, límites de uso gratuito y modelos de cobro. Para la exposición se trabajó específicamente con Google Workspace, enfocándose en la Gmail API, que permite acceder y administrar información de una cuenta de correo mediante solicitudes HTTP.

Durante la exposición se explicaron y probaron los siguientes endpoints:

1. Consultar el perfil de Gmail – GET /gmail/v1/users/me/profile
   Se utilizó este endpoint para obtener información de la cuenta autenticada, incluyendo la dirección de correo electrónico, la cantidad total de mensajes y la cantidad de conversaciones. La API respondió con el código 200 OK, devolviendo los datos solicitados en formato JSON.

2. Listar mensajes – GET /gmail/v1/users/me/messages
   Se realizó una consulta para obtener el listado de mensajes disponibles en la cuenta. La respuesta fue 200 OK y devolvió una colección de identificadores de mensajes (id) y conversaciones (threadId). También se revisaron parámetros que permiten aplicar filtros, limitar resultados y realizar consultas paginadas.

3. Consultar un mensaje específico – GET /gmail/v1/users/me/messages/{id}
   Se utilizó el identificador de un mensaje obtenido previamente para consultar su información detallada. La API respondió con 200 OK, mostrando información del correo, incluyendo sus encabezados y metadatos.

Para realizar estas pruebas se utilizó la herramienta interactiva de Google API Explorer, que permite ejecutar solicitudes directamente desde la documentación. También se revisó el mecanismo de autenticación **OAuth 2.0**, necesario para autorizar el acceso a los datos de una cuenta de Gmail mediante los permisos correspondientes.

Posteriormente, se realizaron pruebas con **Google Maps Platform**, utilizando Postman para consumir servicios relacionados con ubicaciones, direcciones y distancias.

4. **Consultar una ubicación – GET /maps/api/geocode/json**
   Se utilizó la Geocoding API para consultar una dirección ubicada en Quito, Ecuador. La respuesta fue **200 OK** y devolvió información geográfica estructurada, incluyendo los componentes de la dirección, ciudad, provincia y otros datos relacionados con la ubicación.
5. **Calcular distancias – GET /maps/api/distancematrix/json**
   Se utilizó la Distance Matrix API para calcular la distancia y el tiempo estimado de desplazamiento entre una ubicación de origen y un destino cercano a la Universidad Politécnica Salesiana (UPS), utilizando el modo de transporte en vehículo (driving).

La API respondió con **200 OK**, devolviendo una distancia aproximada de **5,8 kilómetros** y un tiempo estimado de **11 minutos** para el recorrido consultado.

Finalmente, durante la práctica se analizaron las diferencias entre las APIs de Google Workspace y Google Maps Platform, especialmente en sus funcionalidades, mecanismos de autenticación y modelos de consumo. Se revisó que Gmail API utiliza permisos OAuth 2.0 para acceder a información del usuario, mientras que los servicios de Google Maps utilizados en las pruebas requieren una API Key y están sujetos a cuotas de uso y facturación según el servicio.

La práctica permitió comprender cómo diferentes servicios de Google pueden integrarse en aplicaciones mediante APIs REST, utilizando solicitudes HTTP, parámetros de consulta y respuestas en formato JSON.

Captura de Pantallas Funcionales:

Gmail API > users > getProfile

¿Te resultó útil? Submit feedback

Method: gmail.users.getProfile

Gets the current user's Gmail profile.

For more details, open the reference documentation.

Try this method

Request parameters

userId: me

Show standard parameters

Credentials

Google OAuth 2.0

OAuth 2.0 provides authenticated access to an API. Show scopes

API key

An API key is a unique string that lets you access an API.

Execute

Terms | Privacy Policy

cURL | HTTP | JAVASCRIPT

```
curl \
  'https://gmail.googleapis.com/gmail/v1/users/me/profile?key=[YOUR_API_KEY]' \
  --header 'Authorization: Bearer [YOUR_ACCESS_TOKEN]' \
  --header 'Accept: application/json' \
  --compressed
```

application/json | Raw HTTP Response

200

```
{
  "emailAddress": "niklas0617@gmail.com",
  "messagesTotal": 39,
  "threadsTotal": 38,
  "historyId": "7590344"
}
```

Request parameters

userId: me

includeSpamTrash

labelIds: string

maxResults: integer

pageToken: string

q: string

Show standard parameters

Credentials

Google OAuth 2.0

OAuth 2.0 provides authenticated access to an API. Show scopes

API key

An API key is a unique string that lets you access an API.

Execute

Terms | Privacy Policy

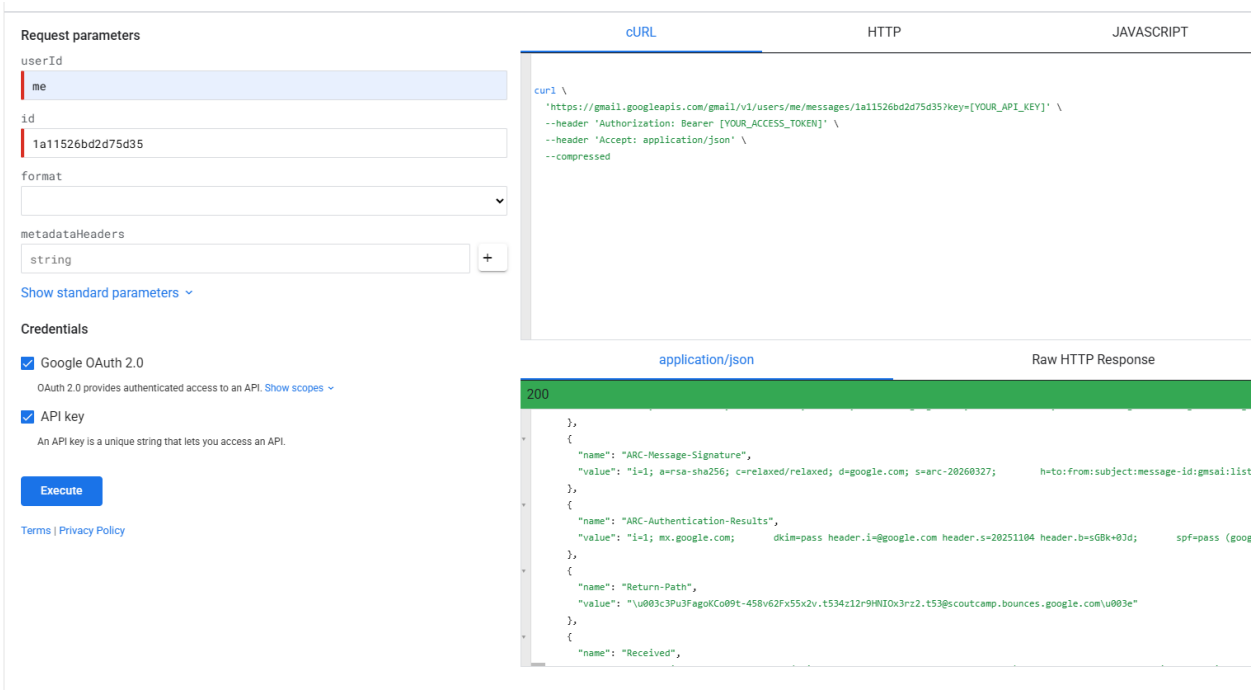
cURL | HTTP | JAVASCRIPT

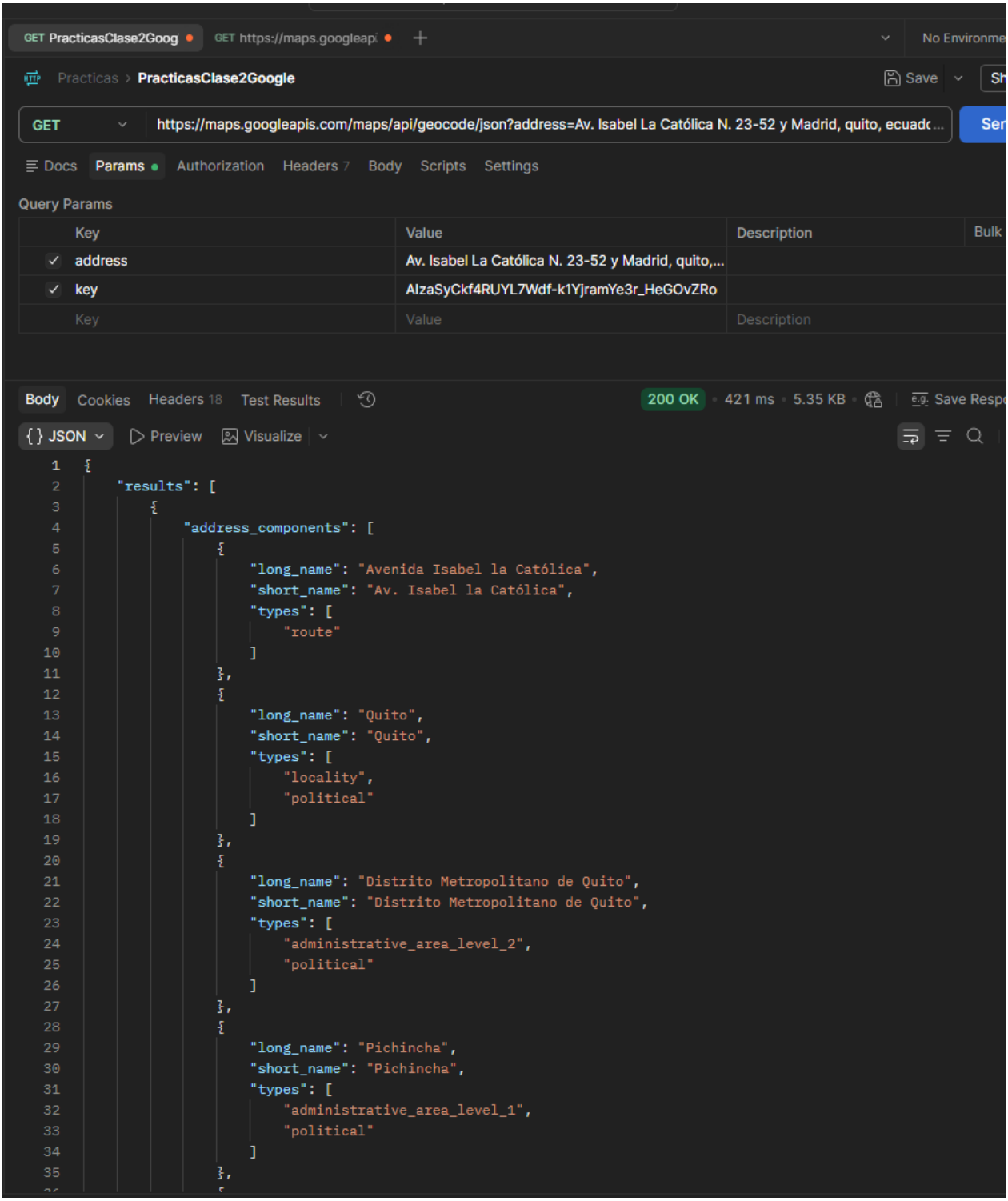
```
curl \
  'https://gmail.googleapis.com/gmail/v1/users/me/messages?key=[YOUR_API_KEY]' \
  --header 'Authorization: Bearer [YOUR_ACCESS_TOKEN]' \
  --header 'Accept: application/json' \
  --compressed
```

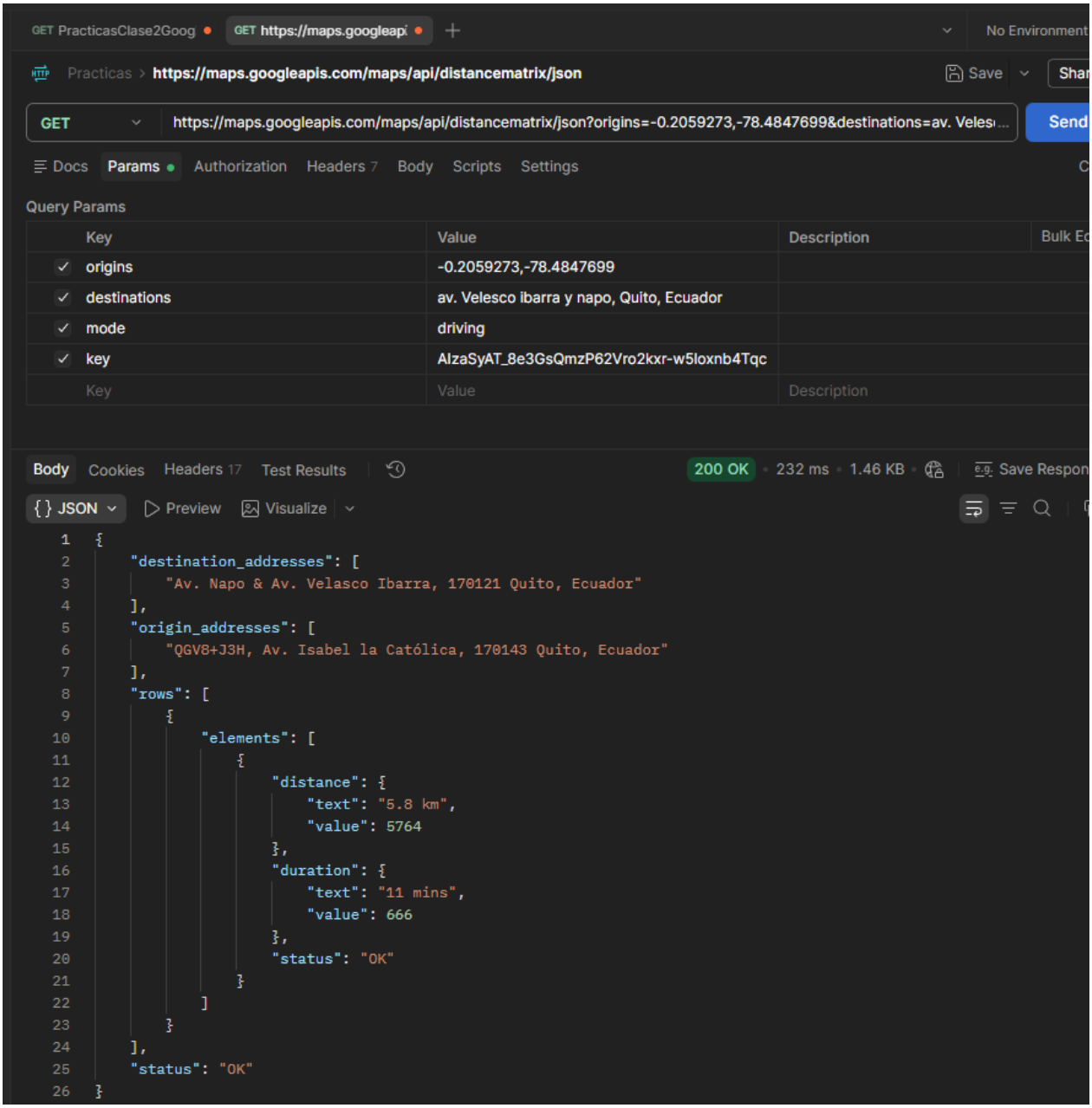
application/json | Raw HTTP Response

200

```
{
  "messages": [
    {
      "id": "1a11526bd2d75d35",
      "threadId": "1a11526bd2d75d35"
    },
    {
      "id": "1a1129f8f7334113",
      "threadId": "1a1129f8f7334113"
    },
    {
      "id": "1a11264ac2c82174",
      "threadId": "1a11264ac2c82174"
    },
    {
```


Request parameters
userId
me
id
1a11526bd2d75d35
format
metadataHeaders
string
Show standard parameters
Credentials
Google OAuth 2.0
OAuth 2.0 provides authenticated access to an API. Show scopes
API key
An API key is a unique string that lets you access an API.
Execute
Terms | Privacy Policy
cURL
HTTP
JAVASCRIPT
curl \
'https://gmail.googleapis.com/gmail/v1/users/me/messages/1a11526bd2d75d35?key=[YOUR_API_KEY]' \
--header 'Authorization: Bearer [YOUR_ACCESS_TOKEN]' \
--header 'Accept: application/json' \
--compressed
application/json
Raw HTTP Response
200
},
{
"name": "ARC-Message-Signature",
"value": "i=1; a=rsa-sha256; c=relaxed/relaxed; d=google.com; s=arc-20260327; h=to:from:subject:message-id:gmsai:list
},
{
"name": "ARC-Authentication-Results",
"value": "i=1; mx.google.com; dkim=pass header.i=@google.com header.s=20251104 header.b=sGBk+0Jd; spf=pass (goog
},
{
"name": "Return-Path",
"value": "\u003c3Pu3FagoKCo09t-458v62Fx55x2v.t534z12r9HNIOx3rz2.t53@scoutcamp.bounces.google.com\u003e"
},
{
"name": "Received",


GET PracticasClase2Goog
GET https://maps.googleap
No Environme
Practicas > PracticasClase2Google
Save
GET
https://maps.googleapis.com/maps/api/geocode/json?address=Av. Isabel La Católica N. 23-52 y Madrid, quito, ecuadc...
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Query Params
Key
Value
Description
Bulk
address
Av. Isabel La Católica N. 23-52 y Madrid, quito,...
key
AIzaSyCkf4RUYL7Wdf-k1YjramYe3r_HeGOvZRo
Key
Value
Description
Body
Cookies
Headers 18
Test Results
200 OK
421 ms
5.35 KB
Save Respo
{} JSON
Preview
Visualize
{
"results": [
{
"address_components": [
{
"long_name": "Avenida Isabel la Católica",
"short_name": "Av. Isabel la Católica",
"types": [
"route"
]
},
{
"long_name": "Quito",
"short_name": "Quito",
"types": [
"locality",
"political"
]
},
{
"long_name": "Distrito Metropolitano de Quito",
"short_name": "Distrito Metropolitano de Quito",
"types": [
"administrative_area_level_2",
"political"
]
},
{
"long_name": "Pichincha",
"short_name": "Pichincha",
"types": [
"administrative_area_level_1",
"political"
]
},



Conclusiones:

Conclusión 1.

La práctica permitió comprender que Google organiza sus APIs según los servicios y funcionalidades que ofrecen, permitiendo a los desarrolladores integrar características específicas sin necesidad de implementarlas desde cero. En el caso de Gmail API, se comprobó cómo consultar información de una cuenta y recuperar mensajes mediante endpoints REST. Además, se identificó

la importancia de utilizar OAuth 2.0 para controlar el acceso a información privada, otorgando únicamente los permisos necesarios para cada operación.

Conclusión 2.

Las pruebas realizadas con Google Maps Platform demostraron cómo una API externa puede resolver operaciones complejas, como obtener información geográfica y calcular distancias y tiempos de recorrido. También se identificó la importancia de considerar los límites de consumo, las cuotas gratuitas disponibles y los costos por utilización al diseñar una integración. Esto permite planificar aplicaciones que aprovechen servicios externos de manera segura, eficiente y con un mejor control de los recursos utilizados.